Capsule-Based RRT-Connect for Dual-Arm Constrained Motion Planning

Algorithm Reference

Overview

Rather than planning directly in the full 12-DOF joint space, this planner operates in **6D task space** (the SE(3) pose of a grasped bar object). A *Capsule* node stores a bar pose together with all feasible dual-arm IK configurations that realise it under the rigid constraint. RRT-Connect grows two trees of Capsules; once a connection is found, an offline **Ladder Graph** search selects the minimum-cost joint-space path.

Key notation:

- $\mathbf{q} \in \mathbb{R}^{12}$: concatenated joint configuration $[\mathbf{q}_L \mid \mathbf{q}_R]$.
- T_A^B: rigid transform of frame A expressed in frame B.
- $\mathcal{C} = (\mathbf{p},\, Q,\, \mathcal{Q},\, \pi)$: a Capsule with bar position $\mathbf{p}$, quaternion Q, config list $\mathcal{Q}$, and parent pointer π.
- FK$(\cdot)$, IK$(\cdot)$: forward/inverse kinematics.
- $d(\mathbf{q}_1, \mathbf{q}_2) = \|\Delta\boldsymbol{\theta}(\mathbf{q}_1, \mathbf{q}_2)\|_2$: joint-space distance with angle wrapping.

Algorithm 1 – Top-Level Planner

Algorithm 1 PLAN($\mathbf{q}_{\text{start}}$, $\mathbf{q}_{\text{goal}}$)

Input: Start config $\mathbf{q}_{\text{start}} \in \mathbb{R}^{12}$, goal config $\mathbf{q}_{\text{goal}} \in \mathbb{R}^{12}$
Output: Joint-space path Π or FAILURE

— Step 1: Initialize global bar←right transform —
Set robot to $\mathbf{q}_{\text{goal}}$
$T_{\text{bar}}^{W} \leftarrow \text{FK}_{\text{bar}}(\mathbf{q}_{\text{goal}})$
$T_{R}^{\text{bar}} \leftarrow \left(T_{\text{bar}}^{W}\right)^{-1} \cdot T_{R}^{W}$ ▷ fixed offset: bar frame → right tool
— Step 2: Compute start and goal bar poses —
$T_{\text{bar},s}^{W} \leftarrow T_{R,s}^{W} \cdot \left(T_{R}^{\text{bar}}\right)^{-1}$
$T_{\text{bar},g}^{W} \leftarrow T_{R,g}^{W} \cdot \left(T_{R}^{\text{bar}}\right)^{-1}$
— Step 3: Generate multiple IK seeds at start and goal —
$\mathcal{Q}_s \leftarrow \textsc{CreateValidConfs}\left(T_{\text{bar},s}^{W},\, T_{R}^{\text{bar}}\right)$
$\mathcal{Q}_g \leftarrow \textsc{CreateValidConfs}\left(T_{\text{bar},g}^{W},\, T_{R}^{\text{bar}}\right)$
— Step 4: Build start and goal Capsules —
$\mathcal{C}_s \leftarrow \textsc{Capsule}\left(T_{\text{bar},s}^{W},\, \mathcal{Q}_s\right)$
$\mathcal{C}_g \leftarrow \textsc{Capsule}\left(T_{\text{bar},g}^{W},\, \mathcal{Q}_g\right)$
— Step 5: RRT-Connect in task space —
$\Pi_c \leftarrow \text{RRTConnectCapsule}(\mathcal{C}_s, \mathcal{C}_g)$
if $\Pi_c = \varnothing$ **then return** FAILURE
end if
— Step 6: Post-process in joint space —
$\Pi \leftarrow \textsc{JointSpaceSmooth}(\Pi_c,\, \text{CSpaceExtendWithProjection})$
$\Pi \leftarrow \textsc{Interpolate}(\Pi,\, \text{CSpaceExtendWithProjection})$
return Π

Algorithm 2 – RRT-Connect in Capsule Space

Algorithm 2 RRTConnectCapsule($\mathcal{C}_s$, $\mathcal{C}_g$)

Input: Start capsule $\mathcal{C}_s$, goal capsule $\mathcal{C}_g$
Output: Ordered joint-config sequence or $\varnothing$

$\mathcal{T}_1 \leftarrow \{\mathcal{C}_s\}$; $\mathcal{T}_2 \leftarrow \{\mathcal{C}_g\}$
for $k = 1$ **to** $N_{\max}$ **do**
  — *Swap: grow from the smaller tree* —
  if $|\mathcal{T}_1| > |\mathcal{T}_2|$ **then swap**$(\mathcal{T}_1, \mathcal{T}_2)$
  end if
  — *Sample a random Capsule in task space* —
  $\mathcal{C}_{\text{rand}} \leftarrow$ Sample() ▷ random bar pose; IK solved later on demand
  — *Extend $\mathcal{T}_1$ toward $\mathcal{C}_{rand}$* —
  $\mathcal{C}_{\text{new}}, \mathcal{T}_1 \leftarrow$ CartExtendToward($\mathcal{T}_1$, $\mathcal{C}_{\text{rand}}$)
  — *Attempt to connect $\mathcal{T}_2$ to $\mathcal{C}_{new}$* —
  $\mathcal{C}_{\text{last}}$, success, $\mathcal{T}_2 \leftarrow$ CartExtendToward($\mathcal{T}_2$, $\mathcal{C}_{\text{new}}$)
  if success **then**
    — *Reconstruct raw capsule path* —
    $\Pi_1 \leftarrow$ Retrace($\mathcal{C}_{\text{new}}$); $\Pi_2 \leftarrow$ Retrace($\mathcal{C}_{\text{last}}$)
    $\Pi_c \leftarrow \Pi_1 \,\|\, \text{reverse}(\Pi_2)$
    $\Pi_c \leftarrow$ Deduplicate(Π_c) ▷ remove duplicate bar poses
    if all capsules in Π_c already expanded **then**
      continue
    end if
    — *Expand each rung to full IK multi-solution set* —
    $\hat{\Pi}_c \leftarrow$ ExpandCartPathLadderGraph(Π_c)
    — *Search ladder graph for best joint-space path* —
    results $\leftarrow$ LadderGraphSearch($\hat{\Pi}_c$)
    if results $= \varnothing$ **then**
      continue
    end if
    return results[0] ▷ minimum-cost joint path
  end if
end for
return $\varnothing$

Algorithm 3 – Extend Toward a Target Capsule

Algorithm 3 CartExtendToward($\mathcal{T}$, $\mathcal{C}_{\text{target}}$)

Input: Tree $\mathcal{T}$, target capsule $\mathcal{C}_{\text{target}}$
Output: Last added capsule $\mathcal{C}_{\text{last}}$, success flag, updated $\mathcal{T}$

$\mathcal{C}_{\text{near}} \leftarrow \arg\min_{\mathcal{C}\in\mathcal{T}} \|\text{CartDist}(\mathcal{C}, \mathcal{C}_{\text{target}})\|_2$
Waypoints $\leftarrow$ CartLinearInterp($\mathcal{C}_{\text{near}}$, $\mathcal{C}_{\text{target}}$) ▷ SLERP + linear position, Alg. 5
$\mathcal{C}_{\text{prev}} \leftarrow \mathcal{C}_{\text{near}}$; all_safe $\leftarrow$ `True`
for each pose T^W_{bar} in Waypoints **do**
    $T^W_R \leftarrow T^W_{\text{bar}} \cdot T^{\text{bar}}_R$
    $\mathbf{q}_R \leftarrow \text{IK}_R\big(T^W_R,\ \text{seed} = \mathcal{C}_{\text{prev}}.\mathcal{Q}[0][6:]\big)$
    if $\mathbf{q}_R \neq \varnothing$ **then**
        $\mathbf{q} \leftarrow$ Project($\mathbf{q}_R$, $\mathcal{C}_{\text{prev}}.\mathcal{Q}[0][:6]$) ▷ Alg. 4
    else
        $\mathbf{q} \leftarrow \varnothing$
    end if
    if $\mathbf{q} = \varnothing$ or InCollision($\mathbf{q}$) **then**
        all_safe $\leftarrow$ `False`;
        break
    end if
    $\mathcal{C}_{\text{new}} \leftarrow \text{Capsule}\big(T^W_{\text{bar}}, [\mathbf{q}], \text{parent} = \mathcal{C}_{\text{prev}}\big)$
    $\mathcal{T} \leftarrow \mathcal{T} \cup \{\mathcal{C}_{\text{new}}\}$; $\mathcal{C}_{\text{prev}} \leftarrow \mathcal{C}_{\text{new}}$
end for
return $\mathcal{C}_{\text{prev}}$, all_safe, $\mathcal{T}$

Algorithm 4 – Dual-Arm Constraint Projection

Algorithm 4 Project($\mathbf{q}_R$, $\mathbf{q}_{L,\text{seed}}$)

Input: Right-arm config $\mathbf{q}_R \in \mathbb{R}^6$, left-arm seed $\mathbf{q}_{L,\text{seed}} \in \mathbb{R}^6$
Output: Full 12-DOF config $\mathbf{q}$ or $\varnothing$

$T^W_R \leftarrow \text{FK}_R(\mathbf{q}_R)$
$T^W_L \leftarrow T^W_R \cdot \big(T^R_L\big)$ ▷ T^R_L is the precomputed rigid offset (constant)
$\mathbf{q}_L \leftarrow \text{IK}_L\big(T^W_L,\ \text{seed} = \mathbf{q}_{L,\text{seed}}\big)$
if $\mathbf{q}_L = \varnothing$ **then**
    return $\varnothing$
end if
return $[\mathbf{q}_L \mid \mathbf{q}_R]$

Algorithm 5 – Cartesian Linear Interpolation (SLERP)

Algorithm 5 CartLinearInterp($\mathcal{C}_1$, $\mathcal{C}_2$)

Input: Capsules $\mathcal{C}_1$, $\mathcal{C}_2$ with poses $(\mathbf{p}_1, Q_1)$ and $(\mathbf{p}_2, Q_2)$
Output: Sequence of waypoint poses

$\Delta p \leftarrow \|\mathbf{p}_2 - \mathbf{p}_1\|$; $\Delta\theta \leftarrow \angle(Q_1, Q_2)$
$N \leftarrow \lceil \max(\Delta p \,/\, r_p,\ \Delta\theta \,/\, r_\theta) \rceil$ ▷ r_p=5cm, r_θ=0.1 rad
if $Q_1 \cdot Q_2 < 0$ **then** $Q_2 \leftarrow -Q_2$
end if ▷ ensure shortest-arc SLERP
Waypoints $\leftarrow [\,]$
for $i = 0$ **to** N **do**
    $t \leftarrow i/N$
    $\mathbf{p}(t) \leftarrow (1-t)\,\mathbf{p}_1 + t\,\mathbf{p}_2$
    $Q(t) \leftarrow \mathrm{Slerp}(Q_1, Q_2, t)$
    Waypoints $\leftarrow$ Waypoints $\cup\ \{(\mathbf{p}(t), Q(t))\}$
end for
return Waypoints

Algorithm 6 – Ladder Graph Construction (ExpandCartPathLadderGraph)

Algorithm 6 ExpandCartPathLadderGraph(Π_c)

Input: Deduplicated capsule sequence $\Pi_c = [\mathcal{C}_0, \ldots, \mathcal{C}_n]$
Output: Expanded ladder graph $\hat{\Pi}_c$ with inter-rung connectivity

— Pass 1: expand each rung to all valid IK solutions —
for each capsule $\mathcal{C}_i$ in Π_c **do**
    if $|\mathcal{C}_i.\mathcal{Q}| \leq 1$ **then**
        $\mathcal{C}_i.\mathcal{Q} \leftarrow$ CreateValidConfs($\mathcal{C}_i$.pose, max_attempts = 20)
    end if
    Append $\mathcal{C}_i$ to $\hat{\Pi}_c$
end for
— Pass 2: build inter-rung connection map —
for $i = 1$ **to** n **do**
    for each config $\mathbf{q}_j \in \hat{\Pi}_c[i].\mathcal{Q}$ **do**
        $\hat{\Pi}_c[i]$.connection$[j] \leftarrow [\,]$
        for each config $\mathbf{q}_k \in \hat{\Pi}_c[i-1].\mathcal{Q}$ **do**
            if $d(\mathbf{q}_j, \mathbf{q}_k) < 2.0$ **then** ▷ threshold in rad
                $\hat{\Pi}_c[i]$.connection$[j]$.append(k)
            end if
        end for
    end for
end for
return $\hat{\Pi}_c$

Algorithm 7 – Ladder Graph Search (LadderGraphSearch)

Algorithm 7 LadderGraphSearch($\hat{\Pi}_c$)

Input: Expanded ladder $\hat{\Pi}_c = [\mathcal{C}_0, \ldots, \mathcal{C}_n]$ with connection maps
Output: All feasible joint-config paths, sorted by total cost (ascending)

Build adjacency E_ℓ: for each rung-pair $(\ell, \ell{+}1)$, $E_\ell[j] \leftarrow \{k \mid k \in \mathcal{C}_{\ell+1}.\text{connection}, \; j \in \mathcal{C}_{\ell+1}.\text{connection}[k]\}$
results $\leftarrow [\,]$
procedure DFS(rung ℓ, index j, path $\mathbf{P}$, cost c)
    if $\ell = n$ **then**
        results.append$\big((\mathbf{P},\ c)\big)$; **return**
    end if
    for each successor $k \in E_\ell[j]$ **do**
        $\delta \leftarrow d(\mathcal{C}_\ell.\mathcal{Q}[j],\ \mathcal{C}_{\ell+1}.\mathcal{Q}[k])$
        DFS($\ell{+}1,\ k,\ \mathbf{P} \cup [\mathcal{C}_{\ell+1}.\mathcal{Q}[k]],\ c + \delta$)
    end for
end procedure
for each start index $j \in \{0, \ldots, |\mathcal{C}_0.\mathcal{Q}| - 1\}$ **do**
    if $E_0[j] \neq \varnothing$ **then**
        DFS($0,\ j,\ [\mathcal{C}_0.\mathcal{Q}[j]],\ 0$)
    end if
end for
sort results by cost ascending
return results

Algorithm 8 – Joint-Space Smoothing

Algorithm 8 JointSpaceSmooth(Π, ExtendFn, N_{iter})

Input: Joint-config path Π, extension function, iteration count N_{iter}
Output: Shorter path (in-place shortcutting)

for $\_ = 1$ **to** N_{iter} **do**
    $(i, j) \leftarrow$ two random indices from $\{0, \ldots, |\Pi| - 1\}$ with $j - i > 1$
    segment $\leftarrow$ CSpaceExtendWithProjection($\Pi[i]$, $\Pi[j]$)
    if segment is valid and collision-free **then**
        $\Pi \leftarrow \Pi[:i] \,\|\, \text{segment} \,\|\, \Pi[j:]$
    end if
end for
return Π

Algorithm 9 – C-Space Extension via Projection

Algorithm 9 CSpaceExtendWithProjection($\mathbf{q}_1$, $\mathbf{q}_2$)

Input: Start config $\mathbf{q}_1$, end config $\mathbf{q}_2$
Output: Sequence of 12-DOF configs interpolating right arm, re-projecting left arm

$\Delta\boldsymbol{\theta}_R \leftarrow \text{AngleDist}(\mathbf{q}_1[6:], \mathbf{q}_2[6:])$
$N \leftarrow \max(1, \lceil \|\Delta\boldsymbol{\theta}_R / r_{\text{joint}}\|_2 \rceil)$ ▷ $r_{\text{joint}} = 5^\circ$ per joint
$\mathbf{q}_{\text{prev}} \leftarrow \mathbf{q}_1$
for $i = 0$ **to** N **do**
    $t \leftarrow i/N$
    $\mathbf{q}_R^{(i)} \leftarrow \text{normalize}(\mathbf{q}_1[6:] + t\,\Delta\boldsymbol{\theta}_R)$
    $\mathbf{q}^{(i)} \leftarrow \textsc{Project}\left(\mathbf{q}_R^{(i)}, \mathbf{q}_{\text{prev}}[:6]\right)$
    if $\mathbf{q}^{(i)} \neq \varnothing$ **then**
        yield $\mathbf{q}^{(i)}$; $\mathbf{q}_{\text{prev}} \leftarrow \mathbf{q}^{(i)}$
    else
        yield $\varnothing$;
        break
    end if
end for

Algorithm 10 – Task-Space Distance

Algorithm 10 CartDist($\mathcal{C}_1$, $\mathcal{C}_2$)

Input: Two Capsules with poses $(R_1, \mathbf{p}_1)$ and $(R_2, \mathbf{p}_2)$
Output: 6D distance vector $\mathbf{d} \in \mathbb{R}^6$

$\Delta\mathbf{p} \leftarrow \mathbf{p}_2 - \mathbf{p}_1$
$\alpha_x \leftarrow \angle(R_1[:,0], R_2[:,0])$
$\alpha_y \leftarrow \angle(R_1[:,1], R_2[:,1])$
$\alpha_z \leftarrow \angle(R_1[:,2], R_2[:,2])$
return $[\Delta p_x, \Delta p_y, \Delta p_z, \alpha_x, \alpha_y, \alpha_z]$

Design Rationale

Design Choice	Rationale
Plan in task space (bar pose SE(3))	Avoids the combinatorial explosion of 12-DOF joint sampling while naturally encoding the object-centric constraint.
Multiple IK seeds per Capsule rung	Captures all kinematic branches at each Cartesian waypoint, enabling the ladder graph to find a globally smooth joint trajectory.
Lazy rung expansion	Full multi-seed IK is deferred until a topologically promising path is found by RRT-Connect, avoiding wasted computation on discarded branches.
Ladder graph + DFS search	Decouples geometric path topology (found by RRT) from optimal joint-config selection (found by graph search), making each sub-problem tractable independently.
Rigid constraint via projection	The left arm is never sampled independently; it is always solved from the right arm pose, guaranteeing the grasp constraint is satisfied at every configuration in the tree.
C-space shortcut smoothing	Post-RRT shortcutting in joint space (interpolated via right-arm linear interpolation + left-arm reprojection) reduces path length without breaking the dual-arm constraint.

Original Design Intent

This section records the original design sketch from which the implementation was derived. It serves as the reference baseline for the gap analysis that follows.

Core Planner Components

Sample. Draw a random 6-DOF bar pose with respect to the robot base frame. *Do not* solve IK at sampling time — the sampled Capsule carries only a pose, no joint configuration. IK is deferred to the extend step, where a good seed is available from the preceding tree node.

Extend. Cartesian-interpolate between two bar poses. The first waypoint is assumed to already carry a valid IK solution (inherited from the nearest tree node); use that configuration to warm-start the IK solver for the next waypoint (iterative, seed-chained IK). If any waypoint fails IK, stop extending and return only the partial path of waypoints for which a valid joint configuration was found.

Collision. Because each Capsule carries exactly one IK solution, collision checking reduces to: set the robot to that joint configuration and test for self-collision and environment collision. This check is entirely local to the Capsule — it has *no* dependence on the Capsule's parent node in the tree.

Distance. Measure the pose difference between two bar Cartesian poses. A direct weighted sum of position and orientation can be awkward to tune because the two quantities have different units (metres vs. radians). An easier alternative, inspired by practice in reinforcement learning, is to pick a small set of *feature points* on the object (e.g. the corners of a bounding box), compute their world-frame positions for each pose, and use the sum of squared Euclidean distances. This yields a purely metric (metres-only) distance with no unit mismatch.

Ladder Graph (Post-RRT Joint-Space Selection)

Once the task-space RRT-Connect finds an initial path (a sequence of bar poses), the joint-continuity problem is addressed in a second phase:

1. **Expand** each Capsule rung on the path to its full set of IK solutions (multiple IK seeds / delta-angle sweep of the bar pose).
2. **Build** a ladder graph: nodes are IK solutions at each rung; edges connect solutions on adjacent rungs that are within a joint-distance threshold.
3. **Search** the ladder graph (e.g. DFS or shortest-path) for the minimum-cost joint-continuous sequence.

Because the ladder graph is built only for the single extracted path, the number of IK evaluations is much smaller than if multi-IK were computed for every RRT node.

Staged Development Order

1. **Stage 1 — Floating-bar RRT.** Disable IK computation (Capsules carry no joint config) and disable collision checking (collision function always returns `False`, or checks only static

bar-vs-obstacle collisions at a fixed robot configuration). Expected result: a correct RRT planner operating purely in task space. Verify by plotting the RRT tree in the workspace and confirming it fills the reachable region.

2. **Stage 2 — IK on, collision off.** Enable IK computation in the extend function; the planner now builds a path in which every waypoint has a valid dual-arm joint configuration. Collision checking remains disabled. Verify by inspecting the tree's joint configurations and checking that the dual-arm constraint is satisfied at each node.
3. **Stage 3 — IK on, collision on.** Enable full self-collision and environment-collision checking. Compare planning success rates and tree shapes across stages.

Observability and Debugging Goals

- **Workspace tree visualisation.** Plot the RRT tree's Cartesian bar poses in the pybullet scene at each stage so the spatial exploration pattern is visible.
- **Failure distribution analysis.** Run random restarts with a fixed per-attempt time budget and distinct random seeds. Record, for each attempt, whether the failure occurred at the task-space level (RRT did not connect), IK level (no valid config found), or collision level (all paths are blocked). Visualise the failure distribution across attempts to identify the dominant bottleneck.
- **Per-stage comparison.** The three development stages are specifically designed so that differences in tree structure and planning success rate between stages directly reveal the contribution of each constraint (IK reachability, collision avoidance) to overall planning difficulty.

Gap Analysis: Design Intent vs. Student Implementation

This section compares the original design sketch against the current implementation, identifies deviations, and discusses implications for performance and debuggability.

Deviation 1 — Sampling: Matches (minor note)

Intent: Do not solve IK at sample time; samples are pure pose draws.
Implementation: `plan()` passes `enable_ik=False` to `_get_sample_fn`, so sampled Capsules always have `config=[]`. **Status: ✓ correct.**

Minor gap: The `enable_ik` default is `True`, making it easy to accidentally re-enable IK during sampling. There is no single top-level flag that cleanly selects among the three intended development stages.

Deviation 2 — Extend / Partial Path: Matches but Fragile

Intent: On IK failure mid-extension, return the partial Cartesian path that has valid IK solutions and stop.
Implementation: `_get_extend_fn` yields an empty Capsule (`config=[]`) at the failure point, then breaks. `extend_towards_capsule` uses `takewhile(not collision_fn, extend)`, and since `collision_fn` returns `True` for any empty-config Capsule, the takewhile automatically stops before the failure. **Status: ✓ correct outcome.**

Risk: The mechanism is *implicit*: the collision function doubles as a "no-IK sentinel." Any future change to collision semantics could break partial-path logic silently, with no type or assertion to catch it.

Deviation 3 — Collision Check: Matches ✓

Intent: Collision check uses only the single IK config stored in the Capsule; no dependence on the parent node.
Implementation: `collision_fn(c.config[0])` — full self-collision plus environment check, parent-independent. **Status: ✓ correct.**

Note: The floating-body collision mode (check bar pose directly, set robot to a fixed config) is present but commented out. Stage 1 of development was attempted but not exposed as a clean toggle.

Deviation 4 — Distance Metric: Partial Deviation

Intent: Pose difference between bar Cartesian poses. The bounding-box / feature-point alternative (sum of 3-D distances of object corners) was proposed as an easier-to-tune option.
Implementation: A 6-D vector $[\Delta p_x, \Delta p_y, \Delta p_z, \alpha_x, \alpha_y, \alpha_z]$ whose ℓ_2 norm is used for nearest-neighbour queries. **Status: × feature-point variant not implemented.**

Performance impact: Position (metres) and rotation (radians) are mixed with equal weight, so the nearest-neighbour heuristic is biased in a workspace-size-dependent way. When the reachable workspace is small (<1 m), rotation terms dominate; when it is large, translation dominates. Bounding-box feature points would give a purely metric (metres-only) distance that is easier to reason about and tune.

Deviation 5 — Staged Development / Debug Flags: Missing

Intent: Three cleanly switchable stages accessible via a single top-level flag:

1. Floating-bar RRT (no IK, no collision).
2. IK on, collision off.
3. IK on, collision on.

Implementation: `enable_ik` exists per-function, but there is no global `enable_collision` flag, no documented stage enumeration, and the floating-body collision path is commented out. **Status: × missing.**

Impact on debugging: When planning fails it is impossible to isolate whether the failure is at the *task-space* level (RRT cannot connect two bar poses), the *IK* level (poses are reachable but no valid joint config exists), or the *collision* level (a valid path exists in free space but is collision-blocked). Without staged flags, each of these hypotheses requires code changes rather than a flag flip.

Deviation 6 — Failure Distribution Logging: Missing

Intent: For each random restart, log where failure occurred and visualise the distribution across attempts.
Implementation: The `max_attempts` outer loop silently `continue`s on failure with no attribution (RRT vs. ladder-graph) and no seed tracking. **Status: × missing.**

Deviation 7 — Ladder Expansion Uses `delta=0.0`: Bug

Intent: After finding a path, expand each rung to *all* possible IK solutions to give the ladder-graph search the widest choice.
Implementation: `expand()` calls `create_valid_confs(delta=0.0, max_attempts=20)`. With `delta=0`, `np.linspace(0, 0, 20)` produces 20 identical zero-rotation perturbations, so the IK solver is called 20 times on the *same* bar pose. **Status: × effective bug (unintentional no-op delta sweep).**

Performance impact: Rungs receive very few distinct IK configurations (the IK solver may not converge to different branches from the same seed/pose), so inter-rung edges are sparse and the DFS frequently finds zero feasible paths even when joint-continuous paths exist. This causes the planner to discard good task-space paths and restart RRT unnecessarily. Changing to `delta=` $\pi/4$ or similar would explore the IK manifold properly.

Deviation 8 — Capsule Struct is Dual-Purpose

Intent: During planning, each Capsule holds one IK solution (tree node); multi-solution expansion happens only on the extracted path (ladder rung).
Implementation: The same `Capsule` class serves both roles. The `parent` pointer means *tree connectivity* during RRT and *rung adjacency* during ladder search. The only distinction is the "fast-skip" check `len(config) > 1` inside `rrt_connect_capsule`. **Status: ~ works but confusing.**

Impact: Accidental mutation of a tree-node Capsule's config during the expand phase can corrupt the tree. Separate `TreeNode` and `LadderRung` types would make each role explicit and eliminate the fast-skip hack.

Deviation 9 — Ladder Search Inside the RRT Loop

Intent: Two clearly separated phases: (1) RRT finds a path, (2) ladder graph is expanded and searched.
Implementation: `expand_path` and `configs_capsule` run *inside* the RRT iteration loop on every connection event, before any path is confirmed to be the best one. If the DFS fails, the loop restarts RRT. **Status: ˜ functional but expensive and hard to profile.**

Impact: Each RRT connection event pays the full multi-IK expansion cost. Profiling cannot easily separate "time in RRT" from "time in ladder search." The two-phase separation in the original design was meant to keep these costs visible and individually tunable.

Summary

Aspect	Intent	Status	Key risk
No IK at sample time	✓	✓	Easy to regress (default=True)
Partial extend path	✓	✓	Implicit via collision sentinel
Parent-independent collision	✓	✓	—
Pose-based distance (unweighted)	✓	×	Unit mismatch, biased NN
Feature-point distance option	proposed	×	Not implemented
3 staged dev. flags	✓	×	Cannot isolate failure mode
Failure distribution logging	✓	×	No restart statistics
Tree workspace visualisation	✓	˜	draw_fn exists but not systematic
`delta > 0` in ladder expand	implied	×	Sparse ladder, frequent DFS failure
Separate tree/ladder types	implied	˜	Single struct, dual role

Proposed Investigation Pathways

Path A — Fix debug scaffolding first. Add `enable_ik: bool` and `enable_collision: bool` to `plan()`. Restore `floating_collision_fn` as stage-1 collision. Verify the task-space RRT explores the workspace correctly before adding IK and collision constraints.

Path B — Fix the distance metric. Implement a feature-point distance: choose 4 corners of the bar's bounding box, compute their world-frame positions for each Capsule pose, and use the sum of squared 3-D distances. This is purely in metres, eliminates unit mixing, and directly matches the RL intuition cited in the design notes. Compare nearest-neighbour quality and tree-growth shape before and after.

Path C — Fix ladder expansion (`delta`). Change `expand()` to use `delta=np.pi/4` (or expose as a parameter). Measure: (a) mean number of distinct IK solutions per rung, (b) DFS success rate (ladder failures vs. RRT failures). This is the highest-leverage single-line fix.

Path D — Separate tree nodes from ladder rungs. Introduce a `LadderRung` dataclass with an explicit multi-config list and connection map. Produce it only inside `expand_path`, not inside RRT. This eliminates the fast-skip hack and the dual-role Capsule ambiguity.

Path E — Add failure attribution and visualisation. For each attempt in the restart loop, record: did RRT connect? did expand succeed? did DFS find a path? Print a summary after all attempts and, when `use_draw=True`, draw the final extent of both RRT trees in the workspace so failure geometry is visible.